

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool : Pseudocode Writing Task (Algorithm Design)
Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I N. Meghana (192411129) that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Meghana

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. XML Parsing Algorithm

BEGIN

LOAD the XML file

IF the XML file cannot be opened THEN

DISPLAY "Error: Unable to load XML file"

STOP

END IF

PARSE the XML document

GET the root element

DISPLAY "User/Product Details"

FOR each child node under the root element

CHECK whether the node contains required details

IF the node contains child elements THEN

FOR each child element

EXTRACT the element name and value

DISPLAY the element name and value

END FOR

ELSE

EXTRACT the node value

DISPLAY the node name and value

END IF

END FOR

DISPLAY "XML data displayed successfully"

END

Explanation: The algorithm first loads and parses the XML document. It starts from the root and traverses the hierarchical child nodes. Required element values are extracted and displayed in a readable form. The file-loading error is also handled.

2. MVC Workflow

BEGIN

USER sends a request to the web application

CONTROLLER receives the request

IDENTIFY the requested operation

VALIDATE the request data

```
IF the request is invalid THEN
    SELECT an error response/view
    SEND the response to the user
    STOP
END IF
```

CONTROLLER calls the required MODEL operation

```
MODEL connects to the database
MODEL executes the required database operation
MODEL retrieves or updates the required data
MODEL returns the result to the CONTROLLER
```

```
CONTROLLER selects the appropriate VIEW
SEND the model data to the VIEW
```

```
VIEW generates the required web page
VIEW creates the final response
```

```
SERVER sends the response to the USER
END
```

Explanation: In MVC, the Controller receives and manages the request, the Model handles application logic and database operations, and the View presents the result. This separation makes the application easier to develop and maintain.

3. JSP Request Flow

```
BEGIN
    CLIENT sends a request for a JSP page

    WEB SERVER receives the request
    IDENTIFY the requested JSP page

    IF the JSP page is not already compiled THEN
        TRANSLATE JSP into a Servlet
        COMPILE the Servlet
    END IF

    EXECUTE the JSP/Servlet

    IF backend processing is required THEN
        CALL the required Servlet or backend component
        CONNECT to the database if required
```

```
EXECUTE the required operation
GET the result
END IF

GENERATE dynamic HTML content using the result
CREATE the HTTP response

SEND the response from server to CLIENT
CLIENT displays the generated web page
END
```

Explanation: A JSP request begins at the client and reaches the web server. The JSP is translated and compiled into a Servlet when necessary. Backend components or a database may be accessed to obtain data, after which dynamic HTML is generated and returned to the client.

4. Database Retrieval

```
BEGIN
START

LOAD database driver if required
ESTABLISH connection with the database

IF connection fails THEN
    DISPLAY "Database connection failed"
    STOP
END IF

CREATE SQL query
PREPARE the query
EXECUTE the SQL query

IF query execution fails THEN
    DISPLAY "Query execution failed"
    CLOSE database connection
    STOP
END IF

GET the result set

IF result set is empty THEN
    DISPLAY "No records found"
ELSE
    FOR each record in the result set
```

```

        EXTRACT required fields
        DISPLAY the retrieved data
    END FOR
END IF

CLOSE result set
CLOSE query/statement
CLOSE database connection

DISPLAY "Data retrieval completed"
END

```

Explanation: The algorithm establishes a database connection, prepares and executes an SQL query, and processes the returned result set. Each record is displayed, and the database resources are closed after the operation.

5. PHP Validation

```

BEGIN
    USER enters data into the web form
    SUBMIT the form

    RECEIVE form data in PHP
    INITIALIZE an empty error list

    READ name, email, and password

    IF name is empty THEN
        ADD "Name is required" to error list
    END IF

    IF email is empty THEN
        ADD "Email is required" to error list
    ELSE IF email format is invalid THEN
        ADD "Enter a valid email address" to error list
    END IF

    IF password is empty THEN
        ADD "Password is required" to error list
    ELSE IF password does not satisfy the required length/format THEN
        ADD "Password does not meet the required format" to error list
    END IF

    IF error list is not empty THEN

```

```
    DISPLAY the error messages
    ASK the user to correct the input
ELSE
    ACCEPT the form data
    PROCESS/store the valid data
    DISPLAY "Form submitted successfully"
END IF
END
```

Explanation: PHP validation checks whether required fields are filled and whether values follow the required format. Invalid data produces clear error messages, while valid data is accepted for further processing. Server-side validation is important because client-side checks alone are not sufficient.